# tf.recompute_grad Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/recompute_grad

> *Defines a function as a recompute-checkpoint for the tape auto-diff.*

## Function Signatures

```
tf.recompute_grad( f )
y = tf.Variable(1.0)
```

## Code Examples

```
tf.recompute_grad(
f
)
```

```
tf.recompute_grad(
f
)
```

```
y = tf.Variable(1.0)
```

```
y = tf.Variable(1.0)
```

```
def my_function(x):
tf.print('running')
z = x*y
return z
```

```
def my_function(x):
```

```
tf.print('running')
```

## Documentation

Defines a function as a recompute-checkpoint for the tape auto-diff.

### View aliases

Compat aliases for migration

See
Migration guide for
more details.

tf.compat.v1.recompute_grad

Tape checkpointing is a technique to reduce the memory consumption of the auto-diff tape:

- Without tape checkpointing operations and intermediate values are

recorded to the tape for use in the backward pass.

- With tape checkpointing, only the function call and its inputs are

recorded. During back-propagation the recompute_grad custom gradient (tf.custom_gradient) recomputes the function under a localized Tape object. This recomputation of the function during backpropagation performs redundant calculation, but reduces the overall memory usage of the Tape.

Without tape checkpointing operations and intermediate values are
recorded to the tape for use in the backward pass.

With tape checkpointing, only the function call and its inputs are
recorded. During back-propagation the recompute_grad custom gradient (tf.custom_gradient) recomputes the function under a localized Tape object. This recomputation of the function during backpropagation performs redundant calculation, but reduces the overall memory usage of the Tape.

Without recompute_grad, the tape contains all intermitate steps, and no recomputation is performed.

If f was a tf.keras Model or Layer object, methods and attributes such as f.variables are not available on the returned function g. Either keep a reference of f , or use g.__wrapped__ for accessing these variables and methods.

Alternatively, use the __wrapped__ attribute to access the original model object.

## Returns